In [1]:

```
# MantisV2 improvements, part 2: idea 2 (learned pooling) on CAP + idea 3 (LoRA fine-tuning)
# Padmanabh Khunt (26100085), AI for Time Series group project
!pip install -q mantis-tsfm gdown h5py
import os, time, json, zipfile, subprocess, requests, copy, traceback
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from mantis.architecture import MantisV2

device = 'cuda'
DATA, OUT = '/tmp/data', '/kaggle/working'
os.makedirs(DATA, exist_ok=True)
net = MantisV2(device=device, return_transf_layer=2, output_token='combined').from_pretrained('paris-noah/MantisV2')
net.return_transf_layer, net.output_token = 2, 'combined'
net.eval()
print(torch.cuda.get_device_name(0))
rows = []

def save_rows():
    pd.DataFrame(rows).to_csv(f'{OUT}/part2_results.csv', index=False)

def safe(fn):
    try:
        fn()
    except Exception as e:
        traceback.print_exc()
        print('FAILED:', type(e).__name__, e, flush=True)

def record(name, method, res, t0, **extra):
    accs = [r[0] for r in res]
    rows.append(dict(dataset=name, method=method, mean=np.mean(accs), std=np.std(accs), per_seed=json.dumps(accs),
                     epochs=json.dumps([r[1] for r in res]), time_s=round(time.time() - t0), **extra))
```

```
    print(f'{name:16s} {method:34s} {np.mean(accs):.4f} +/- {np.std(acc
s):.4f} {extra} {time.time() - t0:.0f}s', flush=True)
    save_rows()

# Data loaders (same as the reproduction notebook)
def load_ts_archive(name):
    folder = f'{DATA}/{name}'
    if not os.path.exists(folder):
        subprocess.run(['wget', '-q', '-O', f'{DATA}/{name}.zip',
                        f'https://www.timeseriesclassification.com/aeon-t
oolkit/{name}.zip'], check=True)
        with zipfile.ZipFile(f'{DATA}/{name}.zip') as z:
            z.extractall(folder)
    out = {}
    for split in ['TRAIN', 'TEST']:
        path = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts'][0]
        X, y, in_data = [], [], False
        with open(path, encoding='utf-8', errors='ignore') as fh:
            for line in fh:
                line = line.strip()
                if not line or line.startswith('#'):
                    continue
                if line.lower().startswith('@data'):
                    in_data = True
                    continue
                if in_data:
                    *channels, label = line.split(':')
                    X.append([[float(v) if v != '?' else np.nan for v in
ch.split(',')] for ch in channels])
                    y.append(label)
        out[split.lower()] = (np.asarray(X, dtype=np.float32), np.asarray
(y))
    return out

TFC = {'HAR': 19930244, 'Epilepsy': 19930199}

def load_tfc(name):
    folder = f'{DATA}/{name}'
    os.makedirs(folder, exist_ok=True)
    files = requests.get(f'https://api.figshare.com/v2/articles/{TFC[nam
```

```
e]}', timeout=30).json()['files']
    out = {}
    for split in ['train', 'test']:
        path = f'{folder}/{split}.pt'
        if not os.path.exists(path):
            url = [f['download_url'] for f in files if f['name'] == f'{sp
lit}.pt'][0]
            with open(path, 'wb') as fh:
                fh.write(requests.get(url, timeout=600).content)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        if X.ndim == 2:
            X = X[:, None, :]
        out[split] = (X, y)
    return out# This Python 3 environment comes with many helpful analyti
cs libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kag
gle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) wil
l list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) th
at gets preserved as output when you create a version using "Save & Run A
ll"
# You can also write temporary files to /kaggle/temp/, but they won't be
saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like compet
itions, datasets, and models to your session
```

```
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
Tesla T4
```

In [2]:

```
# Resize + compact tokens of all 6 layers
def resize(X):
    X = np.nan_to_num(np.asarray(X, dtype=np.float32))
    return F.interpolate(torch.tensor(X), size=512, mode='linear', align_
corners=False).numpy().astype(np.float16)

@torch.no_grad()
def tokens5(Xr, bs=1024):
    """Xr: resized (N, C, 512). Returns (N, C, 6 layers, 5 tokens, 256) f
loat16: per layer the cls token + the 32
    patch tokens averaged in 4 groups of 8 (all 33 tokens for CAP would n
eed about 78 GB)."""
    N, C, _ = Xr.shape
    flat = torch.tensor(Xr).reshape(N * C, 1, 512)
    out = []
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(flat[i:i + bs].to(device).float())
        h = torch.cat([h, net.transf_unit.cls_token.expand(h.shape[0], 1,
-1)], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            t = h[:, 0]
            layers.append(torch.cat([t[:, 32:], t[:, :32].reshape(-1, 4,
8, 256).mean(2)], 1))
        out.append(torch.stack(layers, 1).half().cpu().numpy())
    return np.concatenate(out).reshape(N, C, 6, 5, 256)
```

In [3]:

```
# CAP: download, resize, tokens (one pass per recording machine)
import gdown, h5py
t0 = time.time()
gdown.download(id='1NFwX2CqLrenWF4az0c6J-OglAoD48PAT', output=f'{DATA}/ca
p.zip', quiet=True)
with zipfile.ZipFile(f'{DATA}/cap.zip') as z:
    z.extractall(f'{DATA}/CAP')
os.remove(f'{DATA}/cap.zip')
h5path = [os.path.join(r, f) for r, _, fs in os.walk(f'{DATA}/CAP') for f
in fs if f.endswith('.h5')][0]
print('CAP ready', f'{time.time() - t0:.0f}s', flush=True)
with h5py.File(h5path, 'r') as h:
    ms = sorted(h.keys())
    N = sum(h[f'{m}/data'].shape[0] for m in ms)
    capX = np.zeros((N, 19, 512), dtype=np.float16)
    capT = np.zeros((N, 19, 6, 5, 256), dtype=np.float16)
    capY, capM = np.zeros(N, dtype=np.int64), np.zeros(N, dtype=np.int64)
    pos = 0
    for k, m in enumerate(ms):
        data, start = h[f'{m}/data'], pos
        for s in range(0, data.shape[0], 2000):
            Xr = resize(np.transpose(data[s:s + 2000], (0, 2, 1)))
            capX[pos:pos + len(Xr)] = Xr
            capT[pos:pos + len(Xr)] = tokens5(Xr)
            pos += len(Xr)
        capY[start:pos] = h[f'{m}/labels'][:].ravel()
        capM[start:pos] = k
        print(m, pos - start, f'{time.time() - t0:.0f}s', flush=True)
capY = LabelEncoder().fit_transform(capY)
perm = np.random.RandomState(0).permutation(N)
SPLITS = {'CAP-ID': (perm[:int(0.6375 * N)], perm[N - int(round(0.25 *
N)):]),
          'CAP-OOD': (np.where(np.isin(capM, [0, 1, 3]))[0], np.where(cap
M == 4)[0])}
print({k: (len(a), len(b)) for k, (a, b) in SPLITS.items()})
```

CAP ready 129s
Machine0 12769 210s
Machine1 4694 237s
Machine2 4732 266s
Machine3 9930 327s
Machine4 8265 377s
{'CAP-ID': (25748, 10098), 'CAP-OOD': (27393, 8265)}

In [4]:

```
# Heads: mixer v2 (from the previous notebook) and the idea 2 pooling hea
d with a mixer v2 head on top
class MixerV2(nn.Module):
    """Per-channel projection, channel dropout, gated cross-channel atten
tion (gate starts at 0)."""
    def __init__(self, C, K, mix, d=64, drop=0.3, chan_drop=0.1, d_in=51
2):
        super().__init__()
        self.mix, self.chan_drop = mix, chan_drop
        self.proj = nn.Sequential(nn.LayerNorm(d_in), nn.Linear(d_in, d))
        if mix:
            self.chan_emb = nn.Parameter(torch.zeros(1, C, d))
            self.norm = nn.LayerNorm(d)
            self.attn = nn.MultiheadAttention(d, 2, dropout=drop, batch_f
irst=True)
            self.gate = nn.Parameter(torch.zeros(1))
        self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d, K))

    def forward(self, Z):
        z = self.proj(Z.float())
        if self.training and self.chan_drop > 0:
            z = z * (torch.rand(z.shape[0], z.shape[1], 1, device=z.devic
e) > self.chan_drop).float()
        if self.mix:
            h = self.norm(z + self.chan_emb)
            z = z + torch.tanh(self.gate) * self.attn(h, h, h, need_weigh
ts=False)[0]
        return self.out(z.flatten(1))

class PoolHead(nn.Module):
    """Idea 2: softmax-weighted mix of the 6 layers + attention pooling o
ver the tokens, then the mixer v2 head."""
    def __init__(self, C, K, mix, L=6, heads=4):
        super().__init__()
        self.layer_w = nn.Parameter(torch.zeros(L))
        self.norm = nn.LayerNorm(256)
        self.query = nn.Parameter(torch.randn(heads, 256) * 0.02)
        self.head = MixerV2(C, K, mix, d_in=heads * 256)

    def forward(self, T):
```

```
        w = torch.softmax(self.layer_w, 0)
        t = (self.norm(T.float()) * w[:, None, None]).sum(2)
        a = torch.softmax(t @ self.query.T / 16, 2)
        return self.head((a.transpose(2, 3) @ t).flatten(2))
```

In [5]:

```
# One training loop for every model (20% of the train split held out for
early stopping)
def fit(model, A, y, tr_idx, te_idx, seed, epochs, patience, bs, lr=1e-3,
amp=False):
    """A: input array indexed by sample (numpy, float16); y: encoded labe
ls."""
    fit_idx, va_idx = train_test_split(tr_idx, test_size=0.2, stratify=y
[tr_idx], random_state=seed)
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=0.05)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    scaler = torch.amp.GradScaler('cuda', enabled=amp)
    batch = lambda idx: torch.from_numpy(A[idx]).to(device)

    def predict(idx):
        model.eval()
        preds = []
        with torch.no_grad(), torch.autocast('cuda', dtype=torch.float16,
enabled=amp):
            for i in range(0, len(idx), 256):
                preds.append(model(batch(idx[i:i + 256])).argmax(1).cpu
().numpy())
        return np.concatenate(preds)

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(fit_idx)
        for i in range(0, len(perm), bs):
            b = perm[i:i + bs]
            with torch.autocast('cuda', dtype=torch.float16, enabled=am
p):
                loss = F.cross_entropy(model(batch(b)), torch.from_numpy
(y[b]).long().to(device), label_smoothing=0.1)
            opt.zero_grad()
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
        sched.step()
        va = np.mean(predict(va_idx) == y[va_idx])
```

```
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.detach().clone() for k, v in model.state_d
ict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return float(np.mean(predict(te_idx) == y[te_idx])), ep + 1

def gate_of(head):
    return round(torch.tanh(head.gate).item(), 3) if head.mix else 0.0
```

In [6]:

```
# Idea 2 on CAP: learned pooling, with and without mixer v2, 3 seeds each
def run_pool_cap():
    for split, (tr, te) in SPLITS.items():
        for mix in (False, True):
            t0, res, gates, lws = time.time(), [], [], []
            for s in (0, 1, 2):
                torch.manual_seed(s)
                np.random.seed(s)
                model = PoolHead(19, int(capY.max()) + 1, mix).to(device)
                res.append(fit(model, capT, capY, tr, te, s, epochs=20, p
atience=5, bs=256))
                gates.append(gate_of(model.head))
                lws.append(torch.softmax(model.layer_w, 0).detach().cpu
().numpy().round(3).tolist())
            record(split, 'idea 2 pooling' + (' + mixer v2' if mix else
''), res, t0, gate=json.dumps(gates),
                   layer_weights=json.dumps(lws[0]))

safe(run_pool_cap)
del capT
```

```
CAP-ID           idea 2 pooling                    0.8261 +/- 0.0021
{'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[0.16899999976158142, 0.
1679999977350235, 0.16699999570846558, 0.15600000321865082, 0.17200000
58412552, 0.16599999368190765]'} 227s
CAP-ID           idea 2 pooling + mixer v2         0.8336 +/- 0.0023
{'gate': '[0.183, 0.158, -0.185]', 'layer_weights': '[0.17200000584125
52, 0.17100000381469727, 0.17100000381469727, 0.1550000011920929, 0.16
899999976158142, 0.16300000250339508]'} 234s
CAP-OOD          idea 2 pooling                    0.7651 +/- 0.0043
{'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[0.17299999296665192, 0.
17399999499320984, 0.17399999499320984, 0.15000000596046448, 0.1679999
977350235, 0.16200000047683716]'} 237s
CAP-OOD          idea 2 pooling + mixer v2         0.7653 +/- 0.0053
{'gate': '[0.165, 0.205, -0.181]', 'layer_weights': '[0.17700000107288
36, 0.17599999904632568, 0.1770000010728836, 0.15000000596046448, 0.16
300000250339508, 0.15800000727176666]'} 250s
```

In [7]:

```
# Idea 3: LoRA on the attention projections of MantisV2 layers 0-2, mixer
v2 head on top
class LoRA(nn.Module):
    """Frozen linear layer + trainable low-rank update B @ A (rank r)."""
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base, self.scale = base, alpha / r
        self.A = nn.Parameter(torch.randn(r, base.in_features, device=bas
e.weight.device) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r, device=ba
se.weight.device))

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale

class LoraModel(nn.Module):
    def __init__(self, C, K, r=8):
        super().__init__()
        self.enc = MantisV2(device=device, return_transf_layer=2, output_
token='combined')
        self.enc.load_state_dict(net.state_dict())
        self.enc.remove_transf_layers()
        for p in self.enc.parameters():
            p.requires_grad = False
        for layer in self.enc.transf_unit.transformer.layers:
            layer.attention.wQKV = LoRA(layer.attention.wQKV, r)
            layer.attention.wO = LoRA(layer.attention.wO, r)
        self.head = MixerV2(C, K, True)

    def forward(self, X):
        B, C, L = X.shape
        return self.head(self.enc(X.reshape(B * C, 1, L).float()).reshape
(B, C, 512))

def lora_params(model):
    return sum(p.numel() for n, p in model.named_parameters() if p.requir
es_grad and not n.startswith('head'))

def run_lora(name, Xr, y, tr, te, **kw):
    t0, res, gates = time.time(), [], []
```

```
    for s in (0, 1, 2):
        torch.manual_seed(s)
        np.random.seed(s)
        model = LoraModel(Xr.shape[1], int(y.max()) + 1).to(device)
        res.append(fit(model, Xr, y, tr, te, s, amp=True, **kw))
        gates.append(gate_of(model.head))
    record(name, 'idea 3 LoRA (r=8) + mixer v2', res, t0, gate=json.dumps
(gates), lora_params=lora_params(model))

def small(name, d):
    (Xa, ya), (Xb, yb) = d['train'], d['test']
    Xr = resize(np.concatenate([Xa, Xb]))
    y = LabelEncoder().fit_transform(np.concatenate([ya, yb]))
    return name, Xr, y, np.arange(len(ya)), np.arange(len(ya), len(ya) +
len(yb))

for name in ['FingerMovements', 'Blink']:
    safe(lambda: run_lora(*small(name, load_ts_archive(name)), epochs=30,
patience=8, bs=32))
for folder, name in [('HAR', 'UCI-HAR'), ('Epilepsy', 'Epilepsy-EEG')]:
    safe(lambda: run_lora(*small(name, load_tfc(folder)), epochs=30, pati
ence=8, bs=32))
for split, (tr, te) in SPLITS.items():
    safe(lambda: run_lora(split, capX, capY, tr, te, epochs=4, patience=
2, bs=64))
print(pd.DataFrame(rows)[['dataset', 'method', 'mean', 'std']].round(4).t
o_string(index=False))
```

```
FingerMovements  idea 3 LoRA (r=8) + mixer v2       0.5967 +/- 0.0262
{'gate': '[0.005, 0.002, -0.001]', 'lora_params': 36864} 136s
Blink            idea 3 LoRA (r=8) + mixer v2       1.0000 +/- 0.0000
{'gate': '[0.052, -0.05, 0.044]', 'lora_params': 36864} 41s
UCI-HAR          idea 3 LoRA (r=8) + mixer v2       0.9246 +/- 0.0054
{'gate': '[-0.156, -0.13, 0.145]', 'lora_params': 36864} 289s
Epilepsy-EEG     idea 3 LoRA (r=8) + mixer v2       0.9575 +/- 0.0011
{'gate': '[0.015, 0.007, 0.0]', 'lora_params': 36864} 10s
CAP-ID           idea 3 LoRA (r=8) + mixer v2       0.8194 +/- 0.0011
{'gate': '[-0.068, -0.055, -0.071]', 'lora_params': 36864} 1163s
CAP-OOD          idea 3 LoRA (r=8) + mixer v2       0.7612 +/- 0.0101
{'gate': '[-0.068, -0.075, -0.073]', 'lora_params': 36864} 1220s
        dataset                        method   mean    std
         CAP-ID               idea 2 pooling 0.8261 0.0021
         CAP-ID    idea 2 pooling + mixer v2 0.8336 0.0023
        CAP-OOD               idea 2 pooling 0.7651 0.0043
        CAP-OOD    idea 2 pooling + mixer v2 0.7653 0.0053
FingerMovements idea 3 LoRA (r=8) + mixer v2 0.5967 0.0262
          Blink idea 3 LoRA (r=8) + mixer v2 1.0000 0.0000
        UCI-HAR idea 3 LoRA (r=8) + mixer v2 0.9246 0.0054
   Epilepsy-EEG idea 3 LoRA (r=8) + mixer v2 0.9575 0.0011
         CAP-ID idea 3 LoRA (r=8) + mixer v2 0.8194 0.0011
        CAP-OOD idea 3 LoRA (r=8) + mixer v2 0.7612 0.0101
```